

```
// EZ Vampir Köylü – tek dosya sürümü.
// Kaynak proje src/ altında modüllere bölünmüştür; bu dosya build-tek.js ile üre
import 'dotenv/config';
import http from 'node:http';
import path from 'node:path';
import crypto, { randomInt, randomUUID } from 'node:crypto';
import { fileURLToPath } from 'node:url';
import express from 'express';
import { Server } from 'socket.io';

// ===== ROLLER =====

// Rol tanımları. Tek kaynak burası; sunucu ve istemci aynı listeyi kullanır.

const TEAM = {
  VAMPIRE: 'vampir',
  VILLAGE: 'koy',
  SOLO: 'solo',
};

const ROLES = {
  koylu: {
    id: 'koylu',
    name: 'Köylü',
    team: TEAM.VILLAGE,
    color: '#4ea8ff',
    glyph: 'drop',
    tagline: 'Köyü kurtarmaya çalış.',
    task: 'Gece yeteneğin yok. Gündüz konuş, dinle, doğru kişiye oy ver.',
    hasNightAction: false,
    // Doldurma rolü: lobide sayı seçilmez, boş koltuklar buna gider.
    fillsRemaining: true,
  },
  vampir: {
    id: 'vampir',
    name: 'Vampir',
    team: TEAM.VAMPIRE,
    color: '#e2405b',
    glyph: 'fangs',
    tagline: 'Gece gizlice avlan.',
    task: 'Her gece diğer vampirlerle bir kurban seçin. Gündüz köylü gibi davran.',
    hasNightAction: true,
    nightPrompt: 'Bu gece kimi avlayacaksınız?',
  }
}
```

```
    canTargetSelf: false,
    canTargetTeammate: false,
    min: 1,
    max: 5,
    default: 2,
},
doktor: {
    id: 'doktor',
    name: 'Doktor',
    team: TEAM.VILLAGE,
    color: '#3ddc84',
    glyph: 'cross',
    tagline: 'Her gece birini koru.',
    task: 'Her gece bir kişiyi seç. O kişi vampir saldırısından kurtulur.',
    hasNightAction: true,
    nightPrompt: 'Bu gece kimi koruyacaksın?',
    canTargetSelf: true,
    min: 0,
    max: 2,
    default: 1,
},
bekci: {
    id: 'bekci',
    name: 'Bekçi',
    team: TEAM.VILLAGE,
    color: '#f0a53a',
    glyph: 'shield',
    tagline: 'Bir oyuncuyu gözle.',
    task: 'Her gece bir kişiyi gözle. O kişinin evine gelenlerin adlarını öğrenir',
    hasNightAction: true,
    nightPrompt: 'Bu gece kimi gözleyeceksin?',
    canTargetSelf: false,
    min: 0,
    max: 2,
    default: 1,
},
kahin: {
    id: 'kahin',
    name: 'Kâhin',
    team: TEAM.VILLAGE,
    color: '#a26bf0',
    glyph: 'eye',
    tagline: 'Bir oyuncunun rolünü öğren.',
    task: 'Her gece bir kişiyi seç. O kişinin gerçek rolünü görürsün.',
    hasNightAction: true,
    nightPrompt: 'Kimin rolünü öğrenmek istiyorsun?',
    canTargetSelf: false,
```

```

    min: 0,
    max: 2,
    default: 1,
  },
  joker: {
    id: 'joker',
    name: 'Joker',
    team: TEAM.SOLO,
    color: '#ff7a3d',
    glyph: 'jester',
    tagline: 'Kendi yolunu çiz.',
    task: 'Köy seni asarsa tek başına kazanırsın. Gece ölürsen kaybedersin.',
    hasNightAction: false,
    min: 0,
    max: 2,
    default: 0,
  },
};

```

// Lobide sayısı ayarlanabilen roller (Köylü hariç hepsi).

```
const CONFIGURABLE_ROLES = Object.values(ROLES).filter((r) => !r.fillRemaining);
```

```

function roleName(id) {
  return ROLES[id]?.name ?? 'Bilinmiyor';
}

```

```

function isVampire(roleId) {
  return ROLES[roleId]?.team === TEAM.VAMPIRE;
}

```

```

function defaultRoleCounts() {
  const counts = {};
  for (const role of CONFIGURABLE_ROLES) counts[role.id] = role.default;
  return counts;
}

```

/**

* Rol dağılımını oyuncu sayısına göre doğrular.

* @returns {{ok: true, villagers: number} | {ok: false, error: string}}

*/

```
function validateRoleCounts(counts, playerCount) {
```

```
  let special = 0;
```

```
  for (const role of CONFIGURABLE_ROLES) {
```

```
    const n = counts[role.id];
```

```
    if (!Number.isInteger(n) || n < role.min || n > role.max) {
```

```
      return { ok: false, error: `${role.name} sayısı ${role.min}-${role.max} ara
```

```

    }
    special += n;
}

const vampires = counts.vampir ?? 0;
if (vampires < 1) {
    return { ok: false, error: 'En az 1 vampir olmalı.' };
}
if (vampires * 2 >= playerCount) {
    return { ok: false, error: `${playerCount} oyuncuda en fazla ${Math.ceil(play
}
if (special > playerCount) {
    return { ok: false, error: `Seçilen roller ${special} kişilik ama oyunda ${pl
}
if (special === playerCount) {
    return { ok: false, error: 'En az 1 köylü kalmalı. Bir rolden azalt.' };
}

return { ok: true, villagers: playerCount - special };
}

/** Oyuncu sayısına göre makul bir varsayılan dağılım önerir. */
function suggestRoleCounts(playerCount) {
    const counts = defaultRoleCounts();
    counts.vampir = playerCount >= 15 ? 4 : playerCount >= 12 ? 3 : 2;
    counts.doktor = 1;
    counts.kahin = 1;
    counts.bekci = playerCount >= 10 ? 1 : 0;
    counts.joker = playerCount >= 11 ? 1 : 0;
    return counts;
}

// ===== OYUN MOTORU =====

const PHASE = {
    LOBBY: 'lobby',
    REVEAL: 'reveal',
    NIGHT: 'night',
    NIGHT_RESULT: 'night_result',
    DAY: 'day',
    VOTE: 'vote',
    VOTE_RESULT: 'vote_result',
    END: 'end',
};

const PHASE_LABEL = {
    lobby: 'Oyuncular bekleniyor',

```

```

    reveal: 'Roller dağıtıldı',
    night: 'Gece',
    night_result: 'Sabah oldu',
    day: 'Tartışma zamanı',
    vote: 'Oylama',
    vote_result: 'Oylar sayıldı',
    end: 'Oyun bitti',
  };

const COLORS = [
  '#b8342f', '#2f6fb8', '#3f9b5c', '#8a4fb5', '#c9861f',
  '#20867f', '#b5457e', '#5b6bb5', '#7a5230', '#4a4f57',
  '#a03d8f', '#357a9c', '#8d9b2f', '#c05a2b', '#6d3f9b', '#2f7a4d',
];

const HATS = ['fedora', 'kapusonlu', 'silindir', 'basortu', 'kasket', 'yok'];

const shuffle = (arr) => {
  const a = [...arr];
  for (let i = a.length - 1; i > 0; i--) {
    const j = randomInt(0, i + 1);
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
};

const pick = (arr) => arr[randomInt(0, arr.length)];

const now = () => Date.now();

function defaultSettings() {
  return {
    minPlayers: 8,
    maxPlayers: 16,
    roles: defaultRoleCounts(),
    timers: { reveal: 10, night: 45, nightResult: 8, day: 120, vote: 45, voteResu
    doctorSelfHeal: true,
    doctorRepeat: false,
    jesterEndsGame: true,
    revealRoleOnDeath: true,
    firstDayTalk: true,
    autoAdvance: true,
  };
}

class Room {
  constructor(code, io, { hostName }) {

```

```

    this.code = code;
    this.io = io;
    this.hostName = hostName;
    this.createdAt = now();

    this.players = new Map(); // id -> player
    this.spectators = new Map(); // socketId -> {name}
    this.settings = defaultSettings();

    this.phase = PHASE.LOBBY;
    this.dayNo = 0;
    this.phaseEndsAt = null;
    this.chat = [];
    this.chatSeq = 0;
    this.log = [];
    this.result = null;

    this.night = this.freshNight();
    this.votes = new Map(); // voterId -> targetId | 'pas'
    this.lastVoteResult = null;
    this.lastNightSummary = [];

    this.timer = setInterval(() => this.tick(), 1000);
}

destroy() {
    clearInterval(this.timer);
}

freshNight() {
    return { actions: new Map(), results: new Map(), visits: new Map() };
}

// ----- oyuncular -----

alivePlayers() {
    return [...this.players.values()].filter((p) => p.alive);
}

seatedPlayers() {
    return [...this.players.values()].sort((a, b) => a.seat - b.seat);
}

host() {
    return [...this.players.values()].find((p) => p.isHost) ?? null;
}

```

```

isNameTaken(name) {
  const key = name.trim().toLocaleLowerCase('tr');
  return [...this.players.values()].some((p) => p.name.toLocaleLowerCase('tr'))
}

nextSeat() {
  const used = new Set([...this.players.values()].map((p) => p.seat));
  for (let i = 1; i <= this.settings.maxPlayers; i++) if (!used.has(i)) return i;
  return null;
}

freeCosmetics() {
  const usedColors = new Set([...this.players.values()].map((p) => p.cosmetic.c));
  const color = COLORS.find((c) => !usedColors.has(c)) ?? pick(COLORS);
  return { color, hat: pick(HATS) };
}

/** Yeniden bağlanma: aynı token varsa eski koltuğa oturt. */
reattach(token, socketId) {
  if (!token) return null;
  const player = [...this.players.values()].find((p) => p.token === token);
  if (!player) return null;
  player.socketId = socketId;
  player.connected = true;
  return player;
}

addPlayer({ name, socketId, kickUser = null }) {
  const clean = String(name ?? '').trim().slice(0, 16);
  if (clean.length < 2) return { error: 'İsim en az 2 karakter olmalı.' };
  if (!/^[p{L}\p{N}_.-]+$/u.test(clean)) return { error: 'İsimde sadece harf, rakam, alt çizgi, nokta, tire olabilir.' };
  if (this.isNameTaken(clean)) return { error: 'Bu isim alınmış, başka bir isim seçiniz.' };
  if (this.phase !== PHASE.LOBBY) return { error: 'Oyun başladı. İzleyici olarak katılabilirsiniz.' };
  if (this.players.size >= this.settings.maxPlayers) return { error: 'Köy dolu.' };

  const seat = this.nextSeat();
  const isHost = this.players.size === 0;
  const player = {
    id: randomUUID(),
    token: randomUUID(),
    name: clean,
    kickUser,
    socketId,
    connected: true,
    seat,
    isHost,
    alive: true,
  };
  this.players.add(player);
  if (kickUser) {
    const kickTarget = [...this.players.values()].find((p) => p.token === kickUser);
    if (kickTarget) {
      kickTarget.socketId = null;
      kickTarget.connected = false;
    }
  }
}

```

```

    roleId: null,
    cosmetic: { ...this.freeCosmetics(), number: seat },
    deathInfo: null,
    lastProtected: null,
    ready: false,
  };

  this.players.set(player.id, player);
  this.system(`${clean} köye geldi.`);
  return { player };
}

removePlayer(playerId) {
  const player = this.players.get(playerId);
  if (!player) return;

  if (this.phase === PHASE.LOBBY) {
    this.players.delete(playerId);
    this.system(`${player.name} köyden ayrıldı.`);
    if (player.isHost) this.promoteNewHost();
  } else {
    player.connected = false;
  }
}

promoteNewHost() {
  const next = this.seatedPlayers()[0];
  if (next) {
    next.isHost = true;
    this.system(`${next.name} artık oyun sahibi.`);
  }
}

transferHost(fromId, toId) {
  const from = this.players.get(fromId);
  const to = this.players.get(toId);
  if (!from?.isHost || !to) return { error: 'Bunu yapamazsın.' };
  from.isHost = false;
  to.isHost = true;
  this.system(`Oyun sahipliği ${to.name} oyuncusuna geçti.`);
  return { ok: true };
}

kickPlayer(hostId, targetId) {
  const host = this.players.get(hostId);
  const target = this.players.get(targetId);
  if (!host?.isHost) return { error: 'Sadece oyun sahibi atabilir.' };

```

```

    if (!target || target.id === host.id) return { error: 'Bu oyuncu atılamaz.' }
    if (this.phase !== PHASE.LOBBY) return { error: 'Oyun sırasında atamazsın.' }
    this.players.delete(targetId);
    this.system(`${target.name} oyundan çıkarıldı.`);
    this.io.to(target.socketId).emit('kicked', { reason: 'Oyun sahibi seni çıkard'
    return { ok: true };
}

```

```

setCosmetic(playerId, patch) {
    const player = this.players.get(playerId);
    if (!player) return { error: 'Oyuncu yok.' };
    if (patch.color && COLORS.includes(patch.color)) player.cosmetic.color = patch.color;
    if (patch.hat && HATS.includes(patch.hat)) player.cosmetic.hat = patch.hat;
    if (Number.isInteger(patch.number) && patch.number >= 1 && patch.number <= 99)
        player.cosmetic.number = patch.number;
    }
    return { ok: true };
}

```

// ----- ayarlar -----

```

updateSettings(hostId, patch) {
    const host = this.players.get(hostId);
    if (!host?.isHost) return { error: 'Sadece oyun sahibi ayarları değiştirebili'
    if (this.phase !== PHASE.LOBBY) return { error: 'Oyun başladıktan sonra ayar'

    if (patch.roles) {
        const roles = { ...this.settings.roles };
        for (const role of CONFIGURABLE_ROLES) {
            if (patch.roles[role.id] === undefined) continue;
            const n = Number(patch.roles[role.id]);
            if (!Number.isInteger(n) || n < role.min || n > role.max) {
                return { error: `${role.name} sayısı ${role.min}-${role.max} arasında ol'
            }
            roles[role.id] = n;
        }
        this.settings.roles = roles;
    }
}

```

```

if (patch.timers) {
    const limits = { reveal: [5, 30], night: [15, 180], nightResult: [3, 30], d
    for (const [key, range] of Object.entries(limits)) {
        if (patch.timers[key] === undefined) continue;
        const n = Number(patch.timers[key]);
        if (!Number.isInteger(n) || n < range[0] || n > range[1]) {
            return { error: `Süre değeri ${range[0]}-${range[1]} saniye arasında ol'
        }
    }
}

```

```

        this.settings.timers[key] = n;
    }
}

for (const key of ['doctorSelfHeal', 'doctorRepeat', 'jesterEndsGame', 'revea
    if (typeof patch[key] === 'boolean') this.settings[key] = patch[key];
}

if (patch.minPlayers !== undefined) {
    const n = Number(patch.minPlayers);
    if (!Number.isInteger(n) || n < 4 || n > 16) return { error: 'En az oyuncu
        this.settings.minPlayers = n;
    }
    if (patch.maxPlayers !== undefined) {
        const n = Number(patch.maxPlayers);
        if (!Number.isInteger(n) || n < 4 || n > 16) return { error: 'En fazla oyun
            if (n < this.players.size) return { error: `Şu an ${this.players.size} oyun
                this.settings.maxPlayers = n;
            }
        if (this.settings.minPlayers > this.settings.maxPlayers) {
            this.settings.minPlayers = this.settings.maxPlayers;
        }

        return { ok: true };
    }

autoBalance(hostId) {
    const host = this.players.get(hostId);
    if (!host?.isHost) return { error: 'Sadece oyun sahibi yapabilir.' };
    this.settings.roles = suggestRoleCounts(Math.max(this.players.size, this.sett
    return { ok: true };
}

roleCheck() {
    return validateRoleCounts(this.settings.roles, this.players.size);
}

// ----- oyun akışı -----

start(hostId) {
    const host = this.players.get(hostId);
    if (!host?.isHost) return { error: 'Sadece oyun sahibi başlatabilir.' };
    if (this.phase !== PHASE.LOBBY) return { error: 'Oyun zaten başladı.' };
    if (this.players.size < this.settings.minPlayers) {
        return { error: `En az ${this.settings.minPlayers} oyuncu gerekli. Şu an ${
    }
    const check = this.roleCheck();

```

```

    if (!check.ok) return { error: check.error };

    this.assignRoles();
    this.log = [];
    this.chat = this.chat.filter((m) => m.channel === 'kick').slice(-40);
    this.system('Roller dağıtıldı. Kimseye rolünü söyleme.');
```

this.gotoPhase(PHASE.REVEAL);

```

    return { ok: true };
}

assignRoles() {
    const pool = [];
    for (const role of CONFIGURABLE_ROLES) {
        for (let i = 0; i < this.settings.roles[role.id]; i++) pool.push(role.id);
    }
    while (pool.length < this.players.size) pool.push('koylu');
```

const shuffled = shuffle(pool);

const players = shuffle(this.seatedPlayers());

```

players.forEach((p, i) => {
    p.roleId = shuffled[i];
    p.alive = true;
    p.deathInfo = null;
    p.lastProtected = null;
});
}

gotoPhase(phase) {
    this.phase = phase;
    const t = this.settings.timers;
```

switch (phase) {

case PHASE.REVEAL:

```

    this.dayNo = 0;
    this.setTimer(t.reveal);
    break;
```

case PHASE.NIGHT:

```

    this.night = this.freshNight();
    this.system(`Gece ${this.nightNo()} başladı. Köy uyuyor.`);
    this.setTimer(t.night);
    break;
```

case PHASE.NIGHT_RESULT:

```

    this.resolveNight();
    this.setTimer(t.nightResult);
    break;
```

```

case PHASE.DAY:
    this.dayNo += 1;
    this.votes.clear();
    this.system(
        this.voteEnabled()
        ? `Gün ${this.dayNo}. Tartışın, sonra oylama var.`
        : `Gün ${this.dayNo}. Tanışma günü, bugün oylama yok.`
    );
    this.setTimer(t.day);
    break;

case PHASE.VOTE:
    this.system('Oylama başladı. Asılmasını istediğin kişiyi seç.');
```

```

    this.setTimer(t.vote);
    break;

case PHASE.VOTE_RESULT:
    this.resolveVote();
    this.setTimer(t.voteResult);
    break;

case PHASE.END:
    this.phaseEndsAt = null;
    break;

default:
    this.phaseEndsAt = null;
}

this.broadcast();
}

nightNo() {
    return Math.max(1, this.dayNo === 0 ? 1 : this.dayNo);
}

voteEnabled() {
    return !(this.settings.firstDayTalk && this.dayNo === 1);
}

setTimer(seconds) {
    this.phaseEndsAt = now() + seconds * 1000;
}

tick() {
    if (!this.phaseEndsAt) return;

```

```

    if (this.settings.autoAdvance) this.maybeSkipAhead();

    if (now() >= this.phaseEndsAt) {
        this.advance();
        return;
    }
    this.broadcast();
}

/** Herkes işini bitirdiyse fazı erken kapat. */
maybeSkipAhead() {
    const remaining = this.phaseEndsAt - now();
    if (remaining <= 3000) return;

    if (this.phase === PHASE.NIGHT) {
        const actors = this.alivePlayers().filter((p) => ROLES[p.roleId]?.hasNightA
        if (actors.length === 0) {
            this.phaseEndsAt = now() + 2000;
            return;
        }
        const done = actors.every((p) => this.night.actions.has(p.id));
        if (done) this.phaseEndsAt = now() + 3000;
    }

    if (this.phase === PHASE.VOTE) {
        const voters = this.alivePlayers();
        if (voters.every((p) => this.votes.has(p.id))) this.phaseEndsAt = now() + 3
    }
}

advance() {
    switch (this.phase) {
        case PHASE.REVEAL:
            this.gotoPhase(this.settings.firstDayTalk ? PHASE.DAY : PHASE.NIGHT);
            break;
        case PHASE.NIGHT:
            this.gotoPhase(PHASE.NIGHT_RESULT);
            break;
        case PHASE.NIGHT_RESULT:
            if (!this.finishIfOver()) this.gotoPhase(PHASE.DAY);
            break;
        case PHASE.DAY:
            this.gotoPhase(this.voteEnabled() ? PHASE.VOTE : PHASE.NIGHT);
            break;
        case PHASE.VOTE:
            this.gotoPhase(PHASE.VOTE_RESULT);

```

```

        break;
    case PHASE.VOTE_RESULT:
        if (!this.finishIfOver()) this.gotoPhase(PHASE.NIGHT);
        break;
    default:
        break;
    }
}

skipPhase(hostId) {
    const host = this.players.get(hostId);
    if (!host?.isHost) return { error: 'Sadece oyun sahibi geçebilir.' };
    if (this.phase === PHASE.LOBBY || this.phase === PHASE.END) return { error: '
    this.phaseEndsAt = now();
    return { ok: true };
}

// ----- gece -----

submitNightAction(playerId, targetId) {
    if (this.phase !== PHASE.NIGHT) return { error: 'Şu an gece değil.' };
    const player = this.players.get(playerId);
    if (!player?.alive) return { error: 'Ölüler oynayamaz.' };

    const role = ROLES[player.roleId];
    if (!role?.hasNightAction) return { error: 'Gece yeteneğin yok.' };

    const target = this.players.get(targetId);
    if (!target?.alive) return { error: 'Geçersiz hedef.' };

    if (target.id === player.id && !this.selfTargetAllowed(player)) {
        return { error: 'Kendini seçemezsin.' };
    }
    if (role.canTargetTeammate === false && target.id !== player.id) {
        if (ROLES[target.roleId]?.team === role.team) return { error: 'Takım arkada
    }
    if (player.roleId === 'doktor' && !this.settings.doctorRepeat && player.lastP
        return { error: 'Aynı kişiyi üst üste koruyamazsın.' };
    }

    this.night.actions.set(player.id, target.id);

    if (player.roleId === 'vampir') {
        this.pushChat({
            channel: 'vampir',
            from: 'sistem',
            name: 'Sistem',

```

```

        text: `${player.name} → ${target.name}`,
        kind: 'system',
    });
}

this.broadcast();
return { ok: true, targetId: target.id };
}

selfTargetAllowed(player) {
    if (player.roleId === 'doktor') return this.settings.doctorSelfHeal;
    return ROLES[player.roleId]?.canTargetSelf === true;
}

resolveNight() {
    const visits = new Map(); // targetId -> Set(visitorId)
    const visit = (targetId, visitorId) => {
        if (!visits.has(targetId)) visits.set(targetId, new Set());
        visits.get(targetId).add(visitorId);
    };

    const actorsOf = (roleId) =>
        this.alivePlayers().filter((p) => p.roleId === roleId && this.night.actions

// 1) Vampirler oy birliğine varır, eşitlikte kura.
const vampTally = new Map();
for (const v of actorsOf('vampir')) {
    const t = this.night.actions.get(v.id);
    visit(t, v.id);
    vampTally.set(t, (vampTally.get(t) ?? 0) + 1);
}
let victimId = null;
if (vampTally.size > 0) {
    const top = Math.max(...vampTally.values());
    const tied = [...vampTally.entries()].filter(([ , n]) => n === top).map(([id
    victimId = tied.length === 1 ? tied[0] : pick(tied);
}

// 2) Doktor korur.
const protectedIds = new Set();
for (const d of actorsOf('doktor')) {
    const t = this.night.actions.get(d.id);
    protectedIds.add(t);
    d.lastProtected = t;
    visit(t, d.id);
}

```

```

// 3) Kâhin rol öğrenir.
for (const k of actorsOf('kahin')) {
  const t = this.night.actions.get(k.id);
  visit(t, k.id);
  const target = this.players.get(t);
  this.night.results.set(k.id, {
    kind: 'kahin',
    text: `${target.name} → ${roleName(target.roleId)}`,
  });
}

// 4) Bekçi dışarıdan izler, ziyaretçi sayılmaz.
const watchers = actorsOf('bekci').map((b) => [b, this.night.actions.get(b.id)

const summary = [];

if (victimId && !protectedIds.has(victimId)) {
  const victim = this.players.get(victimId);
  victim.alive = false;
  victim.deathInfo = { cause: 'vampir', night: this.nightNo() };
  summary.push({
    kind: 'death',
    text: `${victim.name} gece vampirler tarafından öldürüldü.`,
    roleText: this.settings.revealRoleOnDeath ? `Rolü: ${roleName(victim.roleId)}` : '',
    playerId: victim.id,
  });
  this.pushChat({
    channel: 'olu',
    from: 'sistem',
    name: 'Sistem',
    text: `${victim.name} aramıza katıldı.`,
    kind: 'system',
  });
} else if (victimId && protectedIds.has(victimId)) {
  summary.push({ kind: 'save', text: 'Bu gece bir saldırı oldu ama doktor yet'
} else {
  summary.push({ kind: 'quiet', text: 'Sakin bir geceydi. Kimse ölmedi.' });
}

// Bekçi sonuçları en son hesaplanır ki tüm ziyaretler dahil olsun.
for (const [b, t] of watchers) {
  const target = this.players.get(t);
  const seen = [...(visits.get(t) ?? new Set())]
    .filter((id) => id !== b.id)
    .map((id) => this.players.get(id)?.name)
    .filter(Boolean);
  this.night.results.set(b.id, {

```

```

        kind: 'bekci',
        text: seen.length
        ? `${target.name} evine gelenler: ${seen.join(', ')}\`
        : `${target.name} evine kimse gelmedi.\`,
    });
}

this.lastNightSummary = summary;
for (const line of summary) this.system(line.text);
this.night.visits = visits;
}

// ----- oylama -----

submitVote(playerId, targetId) {
    if (this.phase !== PHASE.VOTE) return { error: 'Şu an oylama yok.' };
    const player = this.players.get(playerId);
    if (!player?.alive) return { error: 'Ölüler oy veremez.' };

    if (targetId === 'pas') {
        this.votes.set(playerId, 'pas');
        this.broadcast();
        return { ok: true };
    }

    const target = this.players.get(targetId);
    if (!target?.alive) return { error: 'Geçersiz hedef.' };
    if (target.id === player.id) return { error: 'Kendine oy veremezsin.' };

    this.votes.set(playerId, target.id);
    this.broadcast();
    return { ok: true };
}

resolveVote() {
    const tally = new Map();
    for (const [, targetId] of this.votes) {
        if (targetId === 'pas') continue;
        tally.set(targetId, (tally.get(targetId) ?? 0) + 1);
    }

    if (tally.size === 0) {
        this.lastVoteResult = { kind: 'none', text: 'Kimse oy almadı. Bugün kimse a
        this.system(this.lastVoteResult.text);
        return;
    }
}

```

```

const top = Math.max(...tally.values());
const tied = [...tally.entries()].filter(([, n]) => n === top).map(([id]) =>

if (tied.length > 1) {
    const names = tied.map((id) => this.players.get(id)?.name).filter(Boolean).
    this.lastVoteResult = { kind: 'tie', text: `Oylar eşit çıktı (${names}). Bu
    this.system(this.lastVoteResult.text);
    return;
}

const target = this.players.get(tied[0]);
target.alive = false;
target.deathInfo = { cause: 'oylama', day: this.dayNo };

const roleText = this.settings.revealRoleOnDeath ? ` Rolü: ${roleName(target.
this.lastVoteResult = {
    kind: 'lynch',
    playerId: target.id,
    text: `${target.name} ${top} oyla asıldı.${roleText}`,
};
this.system(this.lastVoteResult.text);
this.pushChat({
    channel: 'olu',
    from: 'sistem',
    name: 'Sistem',
    text: `${target.name} aramıza katıldı.`,
    kind: 'system',
});

if (target.roleId === 'joker' && this.settings.jesterEndsGame) {
    this.endGame('joker', `${target.name} tam istediği gibi asıldı. Joker kazan
}
}

// ----- kazanma -----

checkWin() {
    const alive = this.alivePlayers();
    const vamps = alive.filter((p) => isVampire(p.roleId));
    const rest = alive.filter((p) => !isVampire(p.roleId));

    if (vamps.length === 0) {
        return { winner: TEAM.VILLAGE, text: 'Tüm vampirler temizlendi. Köy kazandı
    }
    if (vamps.length >= rest.length) {
        return { winner: TEAM.VAMPIRE, text: 'Vampirler köyü ele geçirdi. Vampirler
    }

```

```

    return null;
}

finishIfOver() {
    if (this.phase === PHASE.END) return true;
    const win = this.checkWin();
    if (!win) return false;
    this.endGame(win.winner, win.text);
    return true;
}

endGame(winner, text) {
    this.result = {
        winner,
        text,
        roles: this.seatedPlayers().map((p) => ({
            id: p.id,
            name: p.name,
            roleId: p.roleId,
            alive: p.alive,
            cosmetic: p.cosmetic,
        })),
    };
    this.system(text);
    this.gotoPhase(PHASE.END);
}

restart(hostId) {
    const host = this.players.get(hostId);
    if (!host?.isHost) return { error: 'Sadece oyun sahibi yeniden başlatabilir.'

    this.phase = PHASE.LOBBY;
    this.dayNo = 0;
    this.phaseEndsAt = null;
    this.result = null;
    this.votes.clear();
    this.night = this.freshNight();
    this.lastNightSummary = [];
    this.lastVoteResult = null;
    this.log = [];
    this.chat = this.chat.filter((m) => m.channel === 'kick').slice(-40);

    for (const p of this.players.values()) {
        p.alive = true;
        p.roleId = null;
        p.deathInfo = null;
        p.lastProtected = null;
    }
}

```

```

    }
    // Kopmuş oyuncularını lobiye dönerken temizle.
    for (const p of [...this.players.values()]) {
        if (!p.connected) this.players.delete(p.id);
    }

    this.system('Yeni oyun için lobiye dönüldü.');
```

this.broadcast();

```

    return { ok: true };
}

// ----- sohbet -----

pushChat({ channel, from, name, text, kind = 'player', color = null }) {
    const msg = { id: ++this.chatSeq, channel, from, name, text, kind, color, ts:
    this.chat.push(msg);
    if (this.chat.length > 400) this.chat.splice(0, this.chat.length - 400);
    return msg;
}

system(text) {
    this.log.push({ text, ts: now(), day: this.dayNo, phase: this.phase });
    if (this.log.length > 200) this.log.shift();
    this.pushChat({ channel: 'sistem', from: 'sistem', name: 'Sistem', text, kind
}

chatChannelFor(player) {
    if (!player) return null;
    if (!player.alive) return 'olu';
    if (this.phase === PHASE.NIGHT && player.roleId === 'vampir') return 'vampir'
    if (this.phase === PHASE.NIGHT) return null;
    if (this.phase === PHASE.LOBBY || this.phase === PHASE.DAY || this.phase ===
    return null;
}

sendChat(playerId, rawText) {
    const player = this.players.get(playerId);
    if (!player) return { error: 'Oyuncu yok.' };

    const text = String(rawText ?? '').trim().slice(0, 240);
    if (!text) return { error: 'Boş mesaj.' };

    const channel = this.chatChannelFor(player);
    if (!channel) return { error: 'Şu an konuşamazsın.' };

    // Basit spam koruması: saniyede 2 mesaj.
    const recent = this.chat.filter((m) => m.from === player.id && now() - m.ts <
```

```

    if (recent.length >= 2) return { error: 'Biraz yavaş.' };

    this.pushChat({
      channel,
      from: player.id,
      name: player.name,
      text,
      color: player.cosmetic.color,
    });
    this.broadcast();
    return { ok: true };
  }

  /** KICK yayın sohbetinden gelen mesaj (webhook). Oyuna müdahale etmez, sadece
  pushKickMessage({ username, text }) {
    this.pushChat({
      channel: 'kick',
      from: 'kick',
      name: String(username ?? 'izleyici').slice(0, 24),
      text: String(text ?? '').slice(0, 240),
      kind: 'kick',
    });
    this.broadcast();
  }

  visibleChat(player) {
    return this.chat.filter((m) => {
      if (m.channel === 'sistem' || m.channel === 'kick' || m.channel === 'koy')
        if (!player) return false;
      if (m.channel === 'vampir') return player.roleId === 'vampir';
      if (m.channel === 'olu') return !player.alive;
      return false;
    }).slice(-120);
  }

  // ----- durum -----

  timeLeft() {
    if (!this.phaseEndsAt) return null;
    return Math.max(0, Math.ceil((this.phaseEndsAt - now()) / 1000));
  }

  publicPlayer(p, viewer) {
    const revealRole =
      this.phase === PHASE.END ||
      (!p.alive && this.settings.revealRoleOnDeath) ||
      (viewer && viewer.id === p.id) ||

```

```

        (viewer && viewer.roleId === 'vampir' && p.roleId === 'vampir'));

return {
  id: p.id,
  name: p.name,
  seat: p.seat,
  alive: p.alive,
  connected: p.connected,
  isHost: p.isHost,
  cosmetic: p.cosmetic,
  roleId: revealRole ? p.roleId : null,
  deathInfo: p.deathInfo,
  kickUser: p.kickUser,
};
}

/** Gece hedef seçimi için uygun oyuncular. */
targetsFor(player) {
  if (this.phase !== PHASE.NIGHT || !player?.alive) return [];
  const role = ROLES[player.roleId];
  if (!role?.hasNightAction) return [];

  return this.alivePlayers()
    .filter((p) => {
      if (p.id === player.id) return this.selfTargetAllowed(player);
      if (role.canTargetTeammate === false && ROLES[p.roleId]?.team === role.team)
        if (player.roleId === 'doktor' && !this.settings.doctorRepeat && player.l
          return true;
    })
    .map((p) => p.id);
}

stateFor(player, { spectatorName = null } = {}) {
  const viewer = player ?? null;
  const role = viewer ? ROLES[viewer.roleId] : null;

  const voteTally = {};
  if (this.phase === PHASE.VOTE || this.phase === PHASE.VOTE_RESULT) {
    for (const [voterId, targetId] of this.votes) {
      if (targetId === 'pas') continue;
      (voteTally[targetId] ??= []).push(voterId);
    }
  }

  const check = this.phase === PHASE.LOBBY ? this.roleCheck() : { ok: true };

  return {

```

```

code: this.code,
hostName: this.hostName,
phase: this.phase,
phaseLabel: PHASE_LABEL[this.phase],
dayNo: this.dayNo,
nightNo: this.nightNo(),
voteEnabled: this.voteEnabled(),
timeLeft: this.timeLeft(),
settings: this.settings,
roleCheck: check,
playerCount: this.players.size,
players: this.seatedPlayers().map((p) => this.publicPlayer(p, viewer)),
chat: this.visibleChat(viewer),
chatChannel: this.chatChannelFor(viewer),
log: this.log.slice(-30),
votes: voteTally,
myVote: viewer ? this.votes.get(viewer.id) ?? null : null,
nightSummary: this.lastNightSummary,
voteResult: this.lastVoteResult,
result: this.result,
you: viewer
  ? {
      id: viewer.id,
      name: viewer.name,
      seat: viewer.seat,
      alive: viewer.alive,
      isHost: viewer.isHost,
      cosmetic: viewer.cosmetic,
      roleId: viewer.roleId,
      teammates:
        viewer.roleId === 'vampir'
          ? this.seatedPlayers().filter((p) => p.roleId === 'vampir' && p.i
            : [],
      nightPrompt: role?.nightPrompt ?? null,
      canActNow: this.phase === PHASE.NIGHT && viewer.alive && !!role?.hasN
      chosenTarget: this.night.actions.get(viewer.id) ?? null,
      targets: this.targetsFor(viewer),
      privateResult: this.night.results.get(viewer.id) ?? null,
    }
    : null,
  spectator: viewer ? null : { name: spectatorName },
};
}

broadcast() {
  for (const p of this.players.values()) {
    if (!p.connected || !p.socketId) continue;

```

```

        this.io.to(p.socketId).emit('state', this.stateFor(p));
    }
    for (const [socketId, s] of this.spectators) {
        this.io.to(socketId).emit('state', this.stateFor(null, { spectatorName: s.n
    }
    }
}

// ===== KICK =====

const ID_BASE = 'https://id.kick.com';
const API_BASE = 'https://api.kick.com';

// -----
// OAuth 2.1 + PKCE
// -----

const b64url = (buf) => buf.toString('base64url');

function createPkcePair() {
    const verifier = b64url(crypto.randomBytes(48));
    const challenge = b64url(crypto.createHash('sha256').update(verifier).digest())
    return { verifier, challenge };
}

function buildAuthorizeUrl({ clientId, redirectUri, scopes, state, challenge }) {
    const url = new URL('/oauth/authorize', ID_BASE);
    url.searchParams.set('response_type', 'code');
    url.searchParams.set('client_id', clientId);
    url.searchParams.set('redirect_uri', redirectUri);
    url.searchParams.set('scope', scopes.join(' '));
    url.searchParams.set('code_challenge', challenge);
    url.searchParams.set('code_challenge_method', 'S256');
    url.searchParams.set('state', state);
    return url.toString();
}

async function exchangeCode({ clientId, clientSecret, redirectUri, code, verifier
    const body = new URLSearchParams({
        grant_type: 'authorization_code',
        client_id: clientId,
        client_secret: clientSecret,
        redirect_uri: redirectUri,
        code_verifier: verifier,
        code,
    });

```

```

const res = await fetch(`${ID_BASE}/oauth/token`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
  body,
});

if (!res.ok) throw new Error(`Token alınamadı (${res.status}): ${await res.text}`);
return res.json();
}

async function fetchKickUser(accessToken) {
  const res = await fetch(`${API_BASE}/public/v1/users`, {
    headers: { Authorization: `Bearer ${accessToken}` },
  });
  if (!res.ok) throw new Error(`Kullanıcı alınamadı (${res.status})`);
  const data = await res.json();
  return Array.isArray(data?.data) ? data.data[0] : data?.data ?? null;
}

// -----
// Webhook imza doğrulama
// -----

let cachedKey = null;
let cachedKeyAt = 0;
const KEY_TTL = 60 * 60 * 1000;

/** KICK'in webhook imza public key'ini getirir (env varsa onu kullanır). */
async function getPublicKey() {
  if (process.env.KICK_PUBLIC_KEY) {
    return process.env.KICK_PUBLIC_KEY.replace(/\n/g, '\n');
  }
  if (cachedKey && Date.now() - cachedKeyAt < KEY_TTL) return cachedKey;

  const res = await fetch(`${API_BASE}/public/v1/public-key`);
  if (!res.ok) throw new Error(`Public key alınamadı (${res.status})`);
  const data = await res.json();
  const key = data?.data?.public_key ?? data?.public_key;
  if (!key) throw new Error('Public key yanıtı beklenen formatta değil.');
```

```

  cachedKey = key;
  cachedKeyAt = Date.now();
  return key;
}

const seenMessages = new Map(); // messageId -> ts
const REPLAY_WINDOW_MS = 5 * 60 * 1000;
```

```

function rememberMessage(id) {
  const cutoff = Date.now() - REPLAY_WINDOW_MS * 2;
  for (const [key, ts] of seenMessages) if (ts < cutoff) seenMessages.delete(key)
  if (seenMessages.has(id)) return false;
  seenMessages.set(id, Date.now());
  return true;
}

/**
 * KICK webhook isteğini doğrular.
 * İmza: base64( RSA-SHA256( `${messageId}.${timestamp}.${rawBody}` ) )
 * @param {object} headers Express req.headers
 * @param {Buffer} rawBody Ham gövde (JSON.parse edilmemiş olmalı)
 */
async function verifyWebhook(headers, rawBody) {
  const messageId = headers['kick-event-message-id'];
  const timestamp = headers['kick-event-message-timestamp'];
  const signature = headers['kick-event-signature'];

  if (!messageId || !timestamp || !signature) {
    return { ok: false, error: 'İmza başlıkları eksik.' };
  }

  const age = Math.abs(Date.now() - Date.parse(timestamp));
  if (!Number.isFinite(age) || age > REPLAY_WINDOW_MS) {
    return { ok: false, error: 'Zaman damgası geçersiz veya çok eski.' };
  }

  if (!rememberMessage(messageId)) {
    return { ok: false, error: 'Bu mesaj zaten işlendi.', duplicate: true };
  }

  let publicKey;
  try {
    publicKey = await getPublicKey();
  } catch (err) {
    return { ok: false, error: `Public key hatası: ${err.message}` };
  }

  const payload = Buffer.concat([
    Buffer.from(`${messageId}.${timestamp}.`, 'utf8'),
    rawBody,
  ]);

  let valid = false;
  try {

```

```

    valid = crypto.verify(
        'sha256',
        payload,
        { key: publicKey, padding: crypto.constants.RSA_PKCS1_PADDING },
        Buffer.from(signature, 'base64')
    );
} catch (err) {
    return { ok: false, error: `Doğrulama hatası: ${err.message}` };
}

if (!valid) return { ok: false, error: 'İmza eşleşmedi.' };

return { ok: true, eventType: headers['kick-event-type'], messageId };
}

/** chat.message.sent olayından oyuna uygun sade bir mesaj çıkarır. */
function parseChatEvent(body) {
    const username =
        body?.sender?.username ??
        body?.sender?.slug ??
        body?.data?.sender?.username ??
        'izleyici';
    const text = body?.content ?? body?.data?.content ?? '';
    if (!text) return null;
    return { username, text };
}

// ===== SUNUCU =====

const __dirname = path.dirname(fileURLToPath(import.meta.url));

const PORT = Number(process.env.PORT ?? 3000);
const PUBLIC_URL = process.env.PUBLIC_URL ?? `http://localhost:${PORT}`;
const HOST_USERNAME = process.env.HOST_USERNAME ?? 'EMREZL';
const ROOM_CODE = (process.env.ROOM_CODE ?? 'KOY').toUpperCase();

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: false } });

app.set('trust proxy', 1);
app.disable('x-powered-by');

// Tek köy. Yayında tek lobi olması en pratiği.
const room = new Room(ROOM_CODE, io, { hostName: HOST_USERNAME });

// -----

```

```

// KICK webhook - ham gövde şart, imza doğrulaması ham baytlar üzerinden yapılır
// -----

app.post('/webhooks/kick', express.raw({ type: '*/*', limit: '256kb' })), async (r
  const check = await verifyWebhook(req.headers, req.body);

  if (!check.ok) {
    if (check.duplicate) return res.status(200).json({ ok: true, skipped: 'duplic
    console.warn('[kick] webhook reddedildi:', check.error);
    return res.status(401).json({ error: check.error });
  }

  let body;
  try {
    body = JSON.parse(req.body.toString('utf8'));
  } catch {
    return res.status(400).json({ error: 'Gövde JSON değil.' });
  }

  if (check.eventType === 'chat.message.sent') {
    const msg = parseChatEvent(body);
    if (msg) room.pushKickMessage(msg);
  }

  res.status(200).json({ ok: true });
});

app.use(express.json({ limit: '64kb' }));
app.use(express.static(path.join(__dirname, 'public'), { maxAge: '1h' }));

// -----
// KICK OAuth 2.1 (PKCE)
// -----

const oauthStates = new Map(); // state -> {verifier, ts}
const kickTickets = new Map(); // ticket -> {name, ts}
const OAUTH_TTL = 10 * 60 * 1000;

setInterval(() => {
  const cutoff = Date.now() - OAUTH_TTL;
  for (const [key, v] of oauthStates) if (v.ts < cutoff) oauthStates.delete(key);
}, 60_000).unref();

const kickConfigured = () =>
  Boolean(process.env.KICK_CLIENT_ID && process.env.KICK_CLIENT_SECRET && process

app.get('/auth/kick', (req, res) => {

```

```

if (!kickConfigured()) {
  return res.status(503).send('KICK uygulaması ayarlanmamış. .env dosyasına KIC
}

const { verifier, challenge } = createPkcePair();
const state = crypto.randomBytes(16).toString('hex');
oauthStates.set(state, { verifier, ts: Date.now() });

res.redirect(
  buildAuthorizeUrl({
    clientId: process.env.KICK_CLIENT_ID,
    redirectUri: process.env.KICK_REDIRECT_URI,
    scopes: ['user:read', 'chat:read'],
    state,
    challenge,
  })
);
});

app.get('/auth/kick/callback', async (req, res) => {
  const { code, state, error } = req.query;
  if (error) return res.status(400).send(`KICK girişi reddedildi: ${error}`);

  const entry = oauthStates.get(state);
  if (!entry) return res.status(400).send('Giriş isteği geçersiz veya süresi dold
  oauthStates.delete(state);

  try {
    const tokens = await exchangeCode({
      clientId: process.env.KICK_CLIENT_ID,
      clientSecret: process.env.KICK_CLIENT_SECRET,
      redirectUri: process.env.KICK_REDIRECT_URI,
      code,
      verifier: entry.verifier,
    });

    const user = await fetchKickUser(tokens.access_token);
    const name = user?.name ?? user?.username ?? 'kickuser';

    // Access token tarayıcıya gönderilmez; sadece doğrulanmış isim aktarılır.
    const ticket = crypto.randomBytes(16).toString('hex');
    kickTickets.set(ticket, { name, ts: Date.now() });

    res.redirect(`/?kick=${ticket}`);
  } catch (err) {
    console.error('[kick] oauth hatası:', err);
    res.status(500).send('KICK girişi tamamlanamadı.');
```

```

    }
  });

  setInterval(() => {
    const cutoff = Date.now() - OAUTH_TTL;
    for (const [key, v] of kickTickets) if (v.ts < cutoff) kickTickets.delete(key);
  }, 60_000).unref();

  app.get('/api/config', (req, res) => {
    res.json({
      hostName: HOST_USERNAME,
      kickLogin: kickConfigured(),
      publicUrl: PUBLIC_URL,
      colors: COLORS,
      hats: HATS,
      roles: Object.values(ROLES).map((r) => ({
        id: r.id,
        name: r.name,
        team: r.team,
        color: r.color,
        glyph: r.glyph,
        tagline: r.tagline,
        task: r.task,
        hasNightAction: r.hasNightAction,
        fillsRemaining: !!r.fillsRemaining,
        min: r.min ?? 0,
        max: r.max ?? 0,
      })),
    });
  });

  app.get('/healthz', (req, res) => res.json({ ok: true, players: room.players.size

// -----
// Socket.IO
// -----

const reply = (cb, payload) => {
  if (typeof cb === 'function') cb(payload);
};

io.on('connection', (socket) => {
  let playerId = null;

  const me = () => (playerId ? room.players.get(playerId) : null);

  /**

```

```

* Payload'suz emit'lerde callback ilk argüman olarak gelir.
* Son argüman fonksiyonsa onu callback kabul ediyoruz.
*/
const on = (event, fn) => {
  socket.on(event, (...args) => {
    const cb = typeof args[args.length - 1] === 'function' ? args.pop() : null;
    fn(args[0] ?? {}, cb);
  });
};

const guard = (cb, fn) => {
  const player = me();
  if (!player) return reply(cb, { error: 'Önce oyuna katıl.' });
  const out = fn(player);
  reply(cb, out ?? { ok: true });
  room.broadcast();
};

on('join', ({ name, token, kickTicket } = {}, cb) => {
  // 1) Yeniden bağlanma
  const existing = room.reattach(token, socket.id);
  if (existing) {
    playerId = existing.id;
    reply(cb, { ok: true, token: existing.token, playerId: existing.id });
    room.system(`${existing.name} tekrar bağlandı.`);
    room.broadcast();
    return;
  }

  // 2) KICK ile doğrulanmış isim
  let finalName = name;
  let kickUser = null;
  if (kickTicket && kickTickets.has(kickTicket)) {
    finalName = kickTickets.get(kickTicket).name;
    kickUser = finalName;
    kickTickets.delete(kickTicket);
  }

  const result = room.addPlayer({ name: finalName, socketId: socket.id, kickUse

  if (result.error) {
    if (result.spectator) {
      room.spectators.set(socket.id, { name: String(finalName ?? 'izleyici').sl
      reply(cb, { ok: true, spectator: true, note: result.error });
      room.broadcast();
      return;
    }
  }

```

```

    return reply(cb, { error: result.error });
  }

  playerId = result.player.id;
  reply(cb, { ok: true, token: result.player.token, playerId });
  room.broadcast();
});

on('cosmetic', (patch, cb) => guard(cb, (p) => room.setCosmetic(p.id, patch ??
on('settings', (patch, cb) => guard(cb, (p) => room.updateSettings(p.id, patch
on('autoBalance', (_, cb) => guard(cb, (p) => room.autoBalance(p.id)));
on('start', (_, cb) => guard(cb, (p) => room.start(p.id)));
on('skipPhase', (_, cb) => guard(cb, (p) => room.skipPhase(p.id)));
on('restart', (_, cb) => guard(cb, (p) => room.restart(p.id)));
on('kickPlayer', ({ targetId } = {}, cb) => guard(cb, (p) => room.kickPlayer(p.
on('transferHost', ({ targetId } = {}, cb) => guard(cb, (p) => room.transferHos
on('nightAction', ({ targetId } = {}, cb) => guard(cb, (p) => room.submitNightA
on('vote', ({ targetId } = {}, cb) => guard(cb, (p) => room.submitVote(p.id, ta
on('chat', ({ text } = {}, cb) => guard(cb, (p) => room.sendChat(p.id, text)));

socket.on('disconnect', () => {
  room.spectators.delete(socket.id);
  const player = me();
  if (player) {
    player.connected = false;
    player.socketId = null;
    room.removePlayer(player.id);
  }
  room.broadcast();
});
});

server.listen(PORT, () => {
  console.log(`EZ Vampir Köylü → ${PUBLIC_URL}`);
  console.log(`Oyun sahibi: KICK:${HOST_USERNAME}`);
  console.log(kickConfigured() ? 'KICK OAuth aktif.' : 'KICK OAuth kapalı (demo i
});

```